

Codex Olympios

A Greek Myth Guide to Static, Inheritance, Polymorphism, and CSE111 Tracing

A memory-first tracing guide for CSE111 students

Authored by: Farhan Zarif [T-13]

CSE111: Programming Language II

Spring 2026 Edition | Version 1.1 | Revised May 2026

Static, Inheritance, Polymorphism, and Tracing Survival Manual

How To Read This Book

This book is written for the student who feels lost when a Java tracing question starts changing static variables, object fields, parent fields, child fields, and local variables at the same time. If that student is you, good. This book was built for you.

The story is Greek myth. The method is real Java. Every myth metaphor is only a memory hook. The real power comes from understanding exactly where every value lives and exactly when it changes.

Rule. Many tracing examples use public fields only to make memory visible. In class-design questions, prefer private fields unless the fixed driver directly requires public access.

Myth Decoder

Myth Hook	Java Idea	What To Remember
Apollo's prophecy	fixed driver code	read the driver before writing the class
Zeus's thunderbolt	static field	one shared class copy
Hermes's message	reference variable	stores a way to reach an object
Kronos's gate	constructor	object setup happens in order
Athena's table	tracing work	write memory changes, do not guess
Hydra's heads	same-named variables	one name may exist in several memory rooms
Proteus's forms	polymorphism	one parent reference can hold child objects

The Promise Of The Book

After reading and practicing from this guide, a student should be able to:

- design missing Java classes from fixed CSE111 driver code;
- trace static variables, instance variables, arrays, and references;
- follow constructor execution step by step;
- understand inheritance, `super`, overriding, and field hiding;
- solve polymorphism questions involving reference type, object type, casting, compile-time errors, and runtime errors;
- explain why an output happened, not only guess the output.

The Ten Parts

The book follows the ten-part path below. Do not jump to polymorphism before mastering static and inheritance. In CSE111, each later trap is built on earlier traps.

Contents

How To Read This Book	1
Olympian Codex: One-Page Cheat Sheet	3
I The Student's Survival Map	4
Chapter 1: The Oracle Of CSE111 Questions	4
Chapter 2: The Map Of Java Memory	6
Chapter 3: Athena's Tracing Table	8

II	Objects, Constructors, And References	12
Chapter 4:	Classes, Objects, And Hermes The Reference	12
Chapter 5:	Constructors: Kronos Opens The First Gate	13
Chapter 6:	Copy Constructor Traps	15
Chapter 7:	Reference Passing	16
Chapter 8:	Object Arrays And Fixed Capacity	17
III	Encapsulation And Access	18
Chapter 9:	Hades And The Private Fields	18
Chapter 10:	Private Parent Fields	18
Chapter 11:	Hephaestus' Workshop	19
IV	Static	22
Chapter 12:	Zeus And The Shared Thunderbolt	22
Chapter 13:	Static Methods	23
Chapter 14:	Static Counters And IDs	24
Chapter 15:	Static Arrays	24
Chapter 16:	Static Array Trap Pattern	24
V	Inheritance	26
Chapter 17:	Divine Bloodlines	26
Chapter 18:	Parent Constructor Before Child Constructor	27
Chapter 19:	super	27
Chapter 20:	Overriding And Overloading	29
Chapter 21:	toString() And Object Printing	29
VI	The Dangerous Zone: Field Hiding And Method Lookup	30
Chapter 22:	The Hydra Of Same Names	30
Chapter 23:	Static Field Hiding	31
Chapter 24:	this, super, And Class Names	32
Chapter 25:	Local Shadowing	32
Chapter 26:	Increment Traps	32
VII	Polymorphism	34
Chapter 27:	Proteus And The Many Forms	34
Chapter 28:	Method Lookup	35
Chapter 29:	Variable Lookup	36
Chapter 30:	Polymorphic Arrays	36
Chapter 31:	Abstract Classes And Interfaces	37

VIII Casting And Error Traps	39
Chapter 32: Icarus And Dangerous Casting	39
Chapter 33: Method Casting Flow	40
Chapter 34: Variable Casting Flow	40
Chapter 35: Object Reference Traps	41
 IX Master Tracing	 42
Chapter 36: Static And Reference Tracing Pattern	42
Chapter 37: Basic Inheritance Tracing Pattern	43
Chapter 38: Override, Constructor, And Hidden Field Tracing Pattern	45
Chapter 39: The Universal Tracing Checklist	46
 X Practice System	 47
Chapter 40: Easy Coding Drills	47
Chapter 41: Medium Inheritance Drills	48
Chapter 42: Hard Polymorphism Drills	48
Chapter 43: Tracing Drill: Static And Copy Constructor	50
Chapter 44: Inheritance And Hidden Fields	51
Chapter 45: Final Exam Strategy	52
Closing Oath Of Athena	53

Olympian Codex: One-Page Cheat Sheet

Question	Answer	Exam Move
Static field	class memory, one shared copy	write it before objects
Instance field	object memory, one copy per object	draw one row per object
Local variable	method frame, temporary	erase it when method ends
Reference assignment	copies an address, not an object	two references may point to one object
Constructor order	parent first, then child	trace constructors before methods
Method call	compile checks reference type, runtime uses object type for overrides	pass both gates
Field access	selected by reference or casting type	fields are not polymorphic
this	current object	child/current part if names repeat
super	parent part of current object	not a separate object
Casting	changes compile-time viewpoint	actual object must fit at runtime

Rule. The shortest full tracing ritual: write Static, Objects, Local Frame, and Output; then update one dangerous line at a time.

Part I The Student's Survival Map

Chapter 1 The Oracle Of CSE111 Questions

Myth hook. Apollo, the god of prophecy, gives the expected output. The driver code is the prophecy. You are not allowed to rewrite the prophecy. You must build the missing world so the prophecy becomes true.

Imagine that a student enters Apollo's temple with a Java question in hand. The left side of the paper shows driver code. The right side shows output. Apollo does not say, "Write any class you like." He says, "This exact driver must produce this exact output." That is the whole game of CSE111 coding tasks.

So the first skill is not coding. The first skill is reading the prophecy. Every constructor call, every dot, every class-name access, and every printed line is a clue. A weak student often starts writing fields immediately. A strong student pauses and asks:

- Which class names already exist?
- Which class names are missing?
- Which methods are called by object names?
- Which methods are called by class names?
- Which fields must be static because the driver uses the class name?
- Which values must be stored in each object because later methods need them?
- Which output lines happen during construction, and which happen during method calls?

This is why CSE111 questions feel tricky. They are not asking you to invent a program from nothing. They are asking you to reverse-engineer a class from its public behavior. The expected output is like a trail of footprints. If you follow it patiently, the missing class reveals itself.

1.1 The Three Question Types

CSE111 questions usually appear in three forms.

1.1.1 Type 1: Missing Class Design

You are given a driver code and expected output. You must write the missing class or classes.

```
public class Tester {  
    public static void main(String[] args) {  
        Hero h = new Hero("Achilles");  
        h.show();  
    }  
}
```

If the output says:

```
Hero: Achilles
```

then the missing class must contain:

- a constructor that accepts a `String`;
- a field to remember the name;
- a `show()` method that prints exactly the required line.

1.1.2 Type 2: Tracing

You are given complete code. You must write the output. These are the questions with the deepest traps.

1.1.3 Type 3: Polymorphism And Error Tables

You are given parent-child classes and statements such as:

```
Parent p = new Child();
p.show();
System.out.println(p.value);
```

You must decide:

- does the line compile?
- if it compiles, does it run?
- if it runs, which method or variable is used?

1.2 The Fixed Driver Rule

Rule. In CSE111 coding tasks, the driver code is fixed. You do not change method calls to make your class easier. Your class must match the driver.

If the driver says:

```
Student.total_students
Student.averageMarks()
mike.setCodingMarks(35)
```

then your class needs:

- a static variable named `total_students`;
- a static method named `averageMarks()`;
- an instance method named `setCodingMarks(int marks)`.

Exam warning. Many students lose marks not because they do not know Java, but because their method name, return type, print format, or static/non-static choice does not match the driver.

1.3 A Full Reading Example

Look at this tiny driver:

```
public class TempleTester {
    public static void main(String[] args) {
        System.out.println("Visitors: " + TempleVisitor.count);
        TempleVisitor v1 = new TempleVisitor("Nikos");
        v1.offer("Olive oil");
        v1.show();
    }
}
```

Before writing the class, read it like Apollo's prophecy.

1. `TempleVisitor.count` uses the class name, so `count` must be static.
2. `new TempleVisitor("Nikos")` means there must be a constructor with one `String` parameter.
3. `v1.offer("Olive oil")` means `offer` is an instance method with one `String` parameter.
4. `v1.show()` means the object must remember enough information to print later.

If the expected output is:

```
Visitors: 0
Nikos offered Olive oil
Visitor: Nikos
Last Offer: Olive oil
```

then the class should be:

```
class TempleVisitor {
    public static int count = 0;
    public String name;
    public String lastOffer = "None";

    public TempleVisitor(String name) {
        this.name = name;
        count++;
    }

    public void offer(String item) {
        lastOffer = item;
        System.out.println(name + " offered " + item);
    }

    public void show() {
        System.out.println("Visitor: " + name);
        System.out.println("Last Offer: " + lastOffer);
    }
}
```

Notice the order. The first line prints `Visitors: 0` before any object is created. Then the constructor increases `count`, but the driver never prints `count` again. So the output does not show 1. A beginner may try to force it into the output, but the driver does not ask for it.

Chapter 2 The Map Of Java Memory

Myth hook. Mount Olympus has different halls. Zeus keeps the shared thunderbolt in one class hall. Each hero carries personal weapons in their own object chamber. Hermes carries messages from one place to another. Athena writes the tracing table.

If a student learns only one thing from this book, it should be this: Java output comes from memory. Every tracing question is just a story of values moving through memory. The code may look like a storm, but the storm always happens in specific places.

Think of Olympus as divided into rooms. The first room is the class room. Zeus keeps class-level things there: static counters, static arrays, total balances, generated IDs, and shared limits. There is only one copy of these things. The second room is the object room. Every new hero, monster, student, account, ticket, or task gets a private object room for its instance fields. The third room is the method room. Whenever a method runs, Java opens a temporary room for parameters and local variables. When the method ends, that temporary room disappears.

Most tracing mistakes happen when students mix up the rooms. They treat a static variable like an object variable. They treat a local variable like a field. They think a new object resets static memory. They think `b = a` copies an object. None of those are true.

2.1 The Four Places Values Can Live

Place	What To Track
Class Area	Static variables and static arrays. There is one shared copy for the whole class.
Object Area	Instance variables. Each object gets its own copy, except static fields.
Method Frame	Local variables and parameters. These exist only while the method is running.
Output Box	Every <code>System.out.println()</code> writes here immediately.

2.2 The One Question To Ask

Whenever you see a variable, ask:

Where does this variable live?

That one question solves most tracing traps.

```
class A {
    public static int x = 5;
    public int y = 2;
    public void test(int y) {
        int x = 1;
        System.out.println(x + " " + y + " " + this.y + " " + A.x);
    }
}
```

Here:

- `x` inside `test()` means local `x`;
- `y` inside `test()` means parameter `y`;
- `this.y` means the object's instance field;
- `A.x` means the class's static field.

Trap. The same name can appear in many places. Java chooses the closest local name first, then fields, unless you explicitly say `this`, `super`, or the class name.

2.3 The Memory Story Of One Line

Take this line:

```
Hero h = new Hero("Perseus");
```

Students often say, “`h` is an object.” That sentence is close, but not exact. The exact story is:

1. Java sees `new Hero("Perseus")` and creates a new `Hero` object in object memory.
2. Java runs the constructor and fills that object's fields.
3. Java creates or uses the reference variable `h`.
4. `h` stores a reference to the object. It is like Hermes carrying the location of the hero.

So `h` is not the whole hero. `h` is the name tag that points to the hero. This one sentence saves students from many reference-passing traps.

2.4 The Three Questions For Every Variable

Whenever you see a variable in a trace, ask these three questions:

1. Is it local to the current method?
2. If not local, is it an instance field of the current object?
3. If not instance, is it static in the class?

Now add inheritance:

1. Is it local?
2. Is it in the child part of the object?
3. Is it in the parent part of the object?
4. Is it static in the child class?
5. Is it static in the parent class?

This is why `x`, `this.x`, `super.x`, `A.x`, and `B.x` can all mean different things. They are not decorative spellings. They are addresses to different rooms.

Chapter 3 Athena's Tracing Table

3.1 The Ritual

Tracing move. Before solving any trace, draw four zones: Static, Objects, Local Frame, Output. Update them after every assignment. Never do the output in your head only.

Step	Static Area	Objects	Output
Start	class variables	none yet	empty
Constructor	update static	create object fields	maybe output
Method call	maybe update static	maybe update object	print immediately

3.2 Tiny Example

```
class Hero {
    public static int count = 0;
    public int power = 5;
    public Hero() {
        count++;
        power += count;
    }
    public void train() {
        power += count;
        System.out.println(power + " " + count);
    }
}

Hero h1 = new Hero();
Hero h2 = new Hero();
h1.train();
h2.train();
```

3.2.1 Step By Step

1. Start: `Hero.count = 0`.
2. `h1 = new Hero()`: `count` becomes 1, `h1.power = 5 + 1 = 6`.
3. `h2 = new Hero()`: `count` becomes 2, `h2.power = 5 + 2 = 7`.
4. `h1.train()`: `h1.power = 6 + 2 = 8`. Print 8 2.
5. `h2.train()`: `h2.power = 7 + 2 = 9`. Print 9 2.

```
8 2
9 2
```

Trap. `count` did not reset for `h2`. Static memory is shared.

3.3 How To Write Workings So You Do Not Get Lost

In many exams, the instruction says the output will not be accepted without workings. That is not just a formality. The workings are the only way to survive a long trace. Athena would not try to remember the whole battle in her head; she would draw the battlefield.

For every trace, make a table like this on rough paper:

Line	Static	Objects	Output
Start	<code>count=0</code>	none	none
<code>new Hero()</code>	<code>count=1</code>	<code>h1.power=6</code>	none
<code>new Hero()</code>	<code>count=2</code>	<code>h2.power=7</code>	none
<code>h1.train()</code>	<code>count=2</code>	<code>h1.power=8</code>	8 2

Do not write only the final value. Write the value after each dangerous line. Dangerous lines are:

- constructor calls;
- method calls;
- assignments with `this`;
- assignments with `super`;
- static updates;
- array element updates;
- `++` or `--`;
- print statements.

3.4 Blank Tracing Templates

Use this small template for short questions:

Static	
Objects	
Local Frame	
Output	

For longer questions, use one row per dangerous line:

Line	Static	Objects / Locals	Output
Start			

3.5 A Beginner's Slow Trace

```

class Trial {
    public static int seed = 3;
    public int score = 2;

    public Trial() {
        score = score + seed;
        seed++;
    }

    public void show(int score) {
        score = score + seed;
        this.score = this.score + score;
        System.out.println(score + " " + this.score + " " + seed);
    }
}

Trial t = new Trial();
t.show(4);

```

Now trace it slowly.

1. Before the driver runs, static `seed = 3`.
2. `new Trial()` creates one object. Its instance `score` starts at 2.
3. Constructor line `score = score + seed`: no local `score` exists in the constructor, so both plain `score` names mean the object's field. New object `score` becomes $2 + 3 = 5$.
4. Constructor line `seed++`: static `seed` becomes 4.
5. `t.show(4)` creates a method frame. Parameter `score = 4`. This parameter hides the instance field.
6. Method line `score = score + seed`: local parameter becomes $4 + 4 = 8$.

7. Method line `this.score = this.score + score`: object field becomes `5 + 8 = 13`.
8. Print local `score`, object `this.score`, and static `seed`.

Output:

```
8 13 4
```

The important lesson is not the final output. The important lesson is why the two `score` variables were different. One lived in the method room. One lived in the object room.

Part II Objects, Constructors, And References

Chapter 4 Classes, Objects, And Hermes The Reference

Myth hook. A class is a divine blueprint. An object is a hero born from that blueprint. A reference is Hermes carrying the address of that hero.

When beginners first learn objects, they often memorize the sentence “an object is an instance of a class.” That sentence is true, but it is not enough for tracing. For tracing, you need to see the scene.

Daedalus designs a blueprint for a bronze guardian. That blueprint is the class. When the forge creates one guardian, that guardian is an object. If the forge creates three guardians, there are three separate objects. Hermes then carries labels telling us where each guardian stands. Those labels are references.

This distinction matters because Java can have many references pointing to one object, and changing the object through one reference is visible through the other references.

4.1 Class Versus Object

```
class Hero {  
    public String name;  
}  
  
Hero h = new Hero();
```

This line has two sides:

- `new Hero()` creates the object;
- `Hero h` creates a reference variable that can point to a `Hero` object.

Rule. A reference variable does not contain the whole object. It contains a way to reach the object.

4.2 Reference Assignment

```
Hero a = new Hero();  
Hero b = a;  
b.name = "Perseus";  
System.out.println(a.name);
```

4.2.1 What Happens

1. `a` points to one `Hero` object.
2. `b = a` means `b` points to the same object.
3. `b.name = "Perseus"` changes that shared object.
4. `a.name` sees the same changed object.

Output:

```
Perseus
```

Trap. `b = a` does not create a copy of the object. It creates another reference to the same object.

4.3 Reference Drawing

Draw this code:

```
Hero a = new Hero();
Hero b = new Hero();
Hero c = a;
```

The drawing is:

Reference	Points To
a	first Hero object
b	second Hero object
c	first Hero object, same as a

If the next line is:

```
c.name = "Perseus";
```

then `a.name` is also `"Perseus"`, because `a` and `c` point to the same object. `b.name` does not change, because `b` points to a different object.

Chapter 5 Constructors: Kronos Opens The First Gate

Myth hook. Kronos controls beginnings. Every object must pass through a constructor before it enters the world. In inheritance, the parent gate opens before the child gate.

5.1 Default And Parameterized Constructors

```
class Hero {
    public String name;
    public int power;

    public Hero() {
        name = "Unknown";
        power = 0;
    }

    public Hero(String n, int p) {
        name = n;
        power = p;
    }
}
```

5.2 Constructor Overloading

Rule. Constructor overloading means multiple constructors with different parameter lists.

```
Hero h1 = new Hero();
Hero h2 = new Hero("Atalanta", 80);
```

Java chooses the constructor by matching the arguments.

5.3 Constructor Chaining With `this()`

```
class Hero {
    public String name;
    public int power;

    public Hero() {
```

```
    this("Unknown", 0);
}

public Hero(String name, int power) {
    this.name = name;
    this.power = power;
}
}
```

Rule. `this(...)` calls another constructor of the same class. It must be the first line inside the constructor.

5.4 The Exact Order When An Object Is Born

For a normal class without inheritance, the order is:

1. allocate the object;
2. give fields their default values;
3. apply field initializer values;
4. run the constructor body.

For example:

```
class Hero {
    public int power = 5;
    public String name;

    public Hero(String name) {
        this.name = name;
        power += 10;
    }
}
```

```
Hero h = new Hero("Theseus");
```

Step by step:

1. Object is allocated.
2. `power` first has default value 0, `name` first has default value `null`.
3. Field initializer changes `power` to 5.
4. Constructor parameter `name` receives "Theseus".
5. `this.name = name` stores "Theseus" in the object field.
6. `power += 10` changes object field `power` from 5 to 15.

Trap. A constructor is not only a place that receives values. It can change static fields, call methods, print output, and mutate other objects. In tracing, constructor lines are full battle lines.

Chapter 6 Copy Constructor Traps

6.1 The Simple Copy

```
class Hero {  
    public int power;  
    public Hero(int power) {  
        this.power = power;  
    }  
    public Hero(Hero other) {  
        this.power = other.power;  
    }  
}
```

This creates a new object with copied values.

6.2 A Copy Constructor With Side Effects

In tracing questions, copy constructors are rarely peaceful. They may copy one value, change a static value, call a method on the old object, or even print output.

```
class ScoreCard {  
    public static int bonus = 5;  
    public int marks = 10;  
    public int total = 0;  
  
    public ScoreCard() {  
        total = marks + bonus;  
        bonus += 2;  
    }  
  
    public ScoreCard(ScoreCard old) {  
        marks = old.total;  
        old.add(3);  
        total = marks + bonus;  
    }  
  
    public void add(int extra) {  
        total += extra + bonus;  
        System.out.println(marks + " " + total + " " + bonus);  
    }  
}
```

If the driver says:

```
ScoreCard a = new ScoreCard();  
ScoreCard b = new ScoreCard(a);
```

then creating `b` does not only fill `b`. It also calls `old.add(3)`, so the old object `a` changes and prints before the copy constructor finishes.

The printed line is:

```
10 25 7
```

because `a.marks` remains 10, `a.total` becomes $15 + 3 + 7 = 25$, and the shared `bonus` is 7 at that moment.

Step	Static	Old object a	New object b
Start	bonus=5	none	none
new ScoreCard()	bonus=7	marks=10, total=15	none
Copy starts	bonus=7	marks=10, total=15	marks=15, total=0
old.add(3)	bonus=7	marks=10, total=25	marks=15, total=0
Copy ends	bonus=7	marks=10, total=25	marks=15, total=22

Trap. The constructor changes the old object by calling `old.add(3)`. So creating the new object can produce output and mutate a previous object.

6.3 How To Trace A Copy Constructor

Tracing move. When you see a constructor such as `new ScoreCard(a)`, pause. Make a new object, but also follow every method call made on the old object.

Checklist:

1. Create the new object.
2. Run field initializers.
3. Run parent constructor if inheritance exists.
4. Run constructor body.
5. If constructor calls another method, trace that method immediately.
6. If that method prints, add output now.

Chapter 7 Reference Passing

Myth hook. Hermes does not carry a whole palace in his bag. He carries directions to the palace. If a method receives an object reference, it can walk to the same object and change it.

```
class Scroll {
    public int marks;
}

class Oracle {
    public static void bless(Scroll s) {
        s.marks += 10;
    }
}

Scroll paper = new Scroll();
paper.marks = 40;
Oracle.bless(paper);
System.out.println(paper.marks);
```

Output:

```
50
```

Rule. Java passes the value of the reference. The method receives a copy of the reference, but both references point to the same object.

7.1 Same Object Passed Twice

```
public void swapLike(Scroll a, Scroll b) {  
    a.marks += 5;  
    b.marks += 7;  
}  
  
swapLike(paper, paper);
```

If `paper.marks` started as 50, it becomes 62. Both `a` and `b` point to the same object.

Exam warning. A trace may call a method like `paper.merge(paper)`. That means `this` and the parameter point to the same object, so one line can update the same object twice.

Chapter 8 Object Arrays And Fixed Capacity

CSE111 coding tasks often use arrays with fixed capacity:

```
class Book {  
    public String title;  
}  
  
class Shelf {  
    public Book[] books = new Book[7];  
    public int count = 0;  
  
    public void addBook(Book b) {  
        if (count < books.length) {  
            books[count] = b;  
            count++;  
        }  
    }  
}
```

Rule. An object array stores references to objects, not copies of objects.

Trap. If two shelves store the same `Book` reference, both shelves are connected to the same book object. A change through one shelf can be visible through the other.

Part III Encapsulation And Access

Chapter 9 Hades And The Private Fields

Myth hook. Hades keeps treasures under the earth. Private fields are hidden from outside classes. You can reach them only through approved gates: methods inside the class.

9.1 private

```
class Account {
    private double balance;
}

Account a = new Account();
// System.out.println(a.balance); // compile-time error
```

Rule. A private field can be accessed directly only inside the class where it is declared.

9.2 Getters And Setters

```
class Account {
    private double balance;

    public double getBalance() {
        return balance;
    }

    public void setBalance(double amount) {
        balance = amount;
    }
}
```

Now outside code can safely interact:

```
Account a = new Account();
a.setBalance(1000);
System.out.println(a.getBalance());
```

Chapter 10 Private Parent Fields

If a parent class has private fields, a child class cannot directly access them.

```
class Product {
    private String title;

    public String getTitle() {
        return title;
    }
}

class Book extends Product {
    public void show() {
        // System.out.println(title); // error
        System.out.println(getTitle());
    }
}
```

```
}
```

Exam warning. Many inheritance coding tasks give parent private fields. The child must use parent getters, setters, or parent methods.

Chapter 11 Hephaestus' Workshop

11.1 Designing From Driver Code

Myth hook. Hephaestus builds exactly what the gods request. In CSE111, the driver code requests the shape of your class. Do not build extra machinery before reading the driver carefully.

11.2 The Five Clues

From the driver, identify:

1. class names;
2. constructor parameter lists;
3. method names and arguments;
4. static versus non-static access;
5. exact output lines and return values.

11.3 Example

```
System.out.println("Open tickets: " + HelpTicket.openTickets);
HelpTicket.printSummary();
HelpTicket t1 = new HelpTicket("Printer jam", 2);
t1.increasePriority(3);
t1.printTicket();
```

This tells us:

- openTickets is static;
- printSummary() is static;
- one constructor takes String, int;
- increasePriority(int) is an instance method;
- printTicket() is an instance method.

Rule. If the driver calls ClassName.method(), the method must be static. If it calls object.method(), the method is normally non-static.

11.4 Full Class Design Walkthrough

Now let us solve a CSE111-style class design question slowly. Suppose Apollo gives this driver:

```
public class ReportTester {
    public static void main(String[] args) {
        System.out.println("Open reports: " + BugReport.openReports);
        System.out.println("Average risk: " + BugReport.averageRisk());
        BugReport b1 = new BugReport("SQL Injection", "High");
        b1.setRiskScore(8);
        b1.details();
        BugReport b2 = new BugReport("Leaked Token", "Critical", 10);
```

```

        b2.details();
        b1.closeReport();
        System.out.println("Open reports: " + BugReport.openReports);
        System.out.println("Average risk: " + BugReport.averageRisk());
    }
}

```

Do not code yet. Read.

1. `BugReport.openReports` is static and public because the driver accesses it directly.
2. `BugReport.averageRisk()` is a static method because it is called by class name.
3. There are two constructors: one takes `String, String`; one takes `String, String, int`.
4. `setRiskScore(8)` means risk score can be changed after construction.
5. `details()` prints object information, so each object must store title, severity, ID, risk score, and status.
6. `closeReport()` changes the status and decreases the number of open reports.

Now decide fields:

Field	Static?	Why
<code>openReports</code>	yes	shared count of all currently open reports
<code>serial</code>	yes	shared ID generator
<code>totalRisk</code>	yes	needed for average risk of all reports
<code>title</code>	no	each report has its own title
<code>severity</code>	no	each report has its own severity
<code>id</code>	no	each report keeps its generated ID
<code>riskScore</code>	no	each report has its own risk score
<code>status</code>	no	each report can be open or closed

Now write the class:

```

class BugReport {
    public static int openReports = 0;
    public static int serial = 0;
    public static int totalRisk = 0;
    public static int totalReports = 0;

    private String title;
    private String severity;
    private int riskScore;
    private String status;
    private String id;

    public BugReport(String title, String severity) {
        this(title, severity, 0);
    }

    public BugReport(String title, String severity, int riskScore) {
        serial++;
        totalReports++;
        openReports++;
        id = "BR-" + serial;
        this.title = title;
        this.severity = severity;
    }
}

```

```
    this.riskScore = riskScore;
    totalRisk += riskScore;
    status = "Open";
}

public void setRiskScore(int riskScore) {
    totalRisk = totalRisk - this.riskScore + riskScore;
    this.riskScore = riskScore;
}

public void closeReport() {
    if (status.equals("Open")) {
        status = "Closed";
        openReports--;
        System.out.println(id + " has been closed");
    }
}

public static double averageRisk() {
    if (totalReports == 0) {
        return 0.0;
    }
    return totalRisk * 1.0 / totalReports;
}

public void details() {
    System.out.println("ID: " + id);
    System.out.println("Title: " + title);
    System.out.println("Severity: " + severity);
    System.out.println("Risk Score: " + riskScore);
    System.out.println("Status: " + status);
}
}
```

The most important line is in `setRiskScore`:

```
totalRisk = totalRisk - this.riskScore + riskScore;
```

Why? Because the object may already have an old risk score. If Zeus's shared total already includes the old value, we must remove the old value before adding the new one. Otherwise the average becomes wrong. This is the kind of real logic hidden behind simple-looking static questions.

Part IV Static

Chapter 12 Zeus And The Shared Thunderbolt

Myth hook. Zeus owns one thunderbolt. Every hero may see it, but there is only one thunderbolt. Static variables work like that.

Static is where many weak students first lose control of tracing. They create `a1`, then `a2`, and they subconsciously imagine that each object receives a fresh copy of every variable. That is true for instance fields. It is false for static fields.

In the myth, Achilles may have his own shield, Odysseus may have his own bow, but Zeus's thunderbolt is not copied for every hero. There is one thunderbolt in Olympus. If anyone changes it, everyone sees the changed thunderbolt. A static variable is that shared thunderbolt.

12.1 Static Variables

```
class Account {
    public static int count = 0;
    public String name;

    public Account(String name) {
        this.name = name;
        count++;
    }
}
```

```
Account a1 = new Account("Abir");
Account a2 = new Account("Mira");
System.out.println(Account.count);
```

Output:

```
2
```

Rule. Static fields belong to the class. They are created once and shared by all objects.

12.2 Static Versus Instance In One Table

```
class Account {
    public static int count = 0;
    public String name;

    public Account(String name) {
        this.name = name;
        count++;
    }
}
```

Member	Where It Lives	How Many Copies
<code>count</code>	class memory	one copy for the whole class
<code>name</code>	object memory	one copy per object

Now run:

```
Account a1 = new Account("Abir");
Account a2 = new Account("Mira");
```

Memory becomes:

- class memory: `Account.count = 2;`
- object a1: `name = "Abir";`
- object a2: `name = "Mira".`

12.3 The Common Wrong Thinking

A beginner may write:

```
a1.count = 1, a2.count = 1
```

That is the wrong mental model. There is no separate count inside a1 and a2. There is one `Account.count`.

Chapter 13 Static Methods

```
class Student {
    public static int total = 0;
    public int marks;

    public static void printTotal() {
        System.out.println(total);
    }
}
```

Rule. A static method can directly use static fields. It cannot directly use instance fields because it does not know which object you mean.

This is illegal:

```
public static void bad() {
    // System.out.println(marks); // error
}
```

This is legal:

```
public static void good(Student s) {
    System.out.println(s.marks);
}
```

13.1 Why Static Methods Cannot Use Instance Fields Directly

Suppose Olympus has three heroes:

```
Student s1 = new Student();
Student s2 = new Student();
Student s3 = new Student();
```

If a static method says:

```
System.out.println(marks);
```

which student's marks should Java print? s1's? s2's? s3's? There is no answer, because a static method belongs to the class, not to a particular object. That is why Java requires an object reference:


```
System.out.println(s1.marks);
```

Now the object is clear.

Chapter 14 Static Counters And IDs

Coding questions often use generated IDs:

```
class BugReport {
    public static int serial = 0;
    public String id;

    public BugReport() {
        serial++;
        id = "BR-" + serial;
    }
}
```

14.1 Trace

1. Before any object: `serial = 0`.
2. First object: `serial` becomes 1, `id` becomes BR-1.
3. Second object: `serial` becomes 2, `id` becomes BR-2.

Trap. The ID should usually be saved inside the object. Do not calculate it again every time from the current serial, or all objects may appear to have the latest ID.

Chapter 15 Static Arrays

```
class LibraryStock {
    public static int[] copies = {3, 3, 3};
    public static String[] titles = {
        "Intro to Java", "Data Structures", "OOP Patterns"
    };
}
```

Rule. A static array is one shared array reference stored in the class area. Changing an element changes the shared array.

15.1 Element Change Versus Array Replacement

```
LibraryStock.copies[0] = 10;           // same array, one element changed
LibraryStock.copies = new int[] {1, 2}; // now points to a new array
```

Exam warning. Static-array traces often use both element mutation and array replacement. Track the array reference, not only the numbers inside it.

Chapter 16 Static Array Trap Pattern

```
class SupplyTrace {
    public static int temp = 10;
    public int sum = 0;
    public static int[] y = {2, 5};

    public SupplyTrace() {
```

```
    temp -= 2;
    sum = temp + y[0];
    y[1] = sum - temp;
  }
}
```

16.1 What Makes It Tricky

- `temp` is static, so both objects share it.
- `y` is a static array, so both objects share the same array.
- `sum` is instance, so each object has its own `sum`.
- The constructor changes static state before any method call in the driver.

Tracing move. For static-heavy traces, write a separate static row before writing object rows. Update the static row every time any object touches a static variable.

16.2 Why Static Traces Feel Difficult

Static traces are not random. They combine four forces:

1. a static primitive such as `temp`;
2. a static array such as `x` or `y`;
3. instance fields such as `sum`;
4. local variables with the same names as fields.

That means one line may contain three different memory rooms:

```
int local = 2 - this.y[0] - SupplyTrace.y[1];
```

Read it slowly:

- left side `int local` creates a local variable;
- `this.y[0]` accesses the array through the object, but if `y` is static, it is still the shared class array;
- `SupplyTrace.y[1]` clearly accesses the class array.

So the line is not “the object field changes.” It is “create a temporary local variable using values from the shared static array.” That is a completely different level of precision.

Part V Inheritance

Chapter 17 Divine Bloodlines

Myth hook. A child of a god inherits divine abilities, but may also gain new powers. In Java, a child class inherits accessible fields and methods from the parent class.

Inheritance should not be taught as “write extends.” That is syntax. The real concept is reuse plus specialization. A parent class contains common behavior. A child class receives that behavior and then adds or changes what is special.

Think of a divine bloodline. A demigod may inherit strength from Ares, wisdom from Athena, or speed from Hermes. But the demigod is still an individual. They may carry their own weapon, have their own mission, and speak in their own voice. In Java, the parent class gives the inherited part, and the child class adds the specialized part.

```
class Shape {
    public String name;
    public String color;

    public void displayInfo() {
        System.out.println("Name: " + name);
        System.out.println("Color: " + color);
    }
}

class Circle extends Shape {
    public double radius;

    public double area() {
        return 3.14 * radius * radius;
    }
}
```

Rule. `extends` means the child class receives the parent class’s accessible members.

17.1 What A Child Object Really Contains

When you create:

```
Circle c = new Circle();
```

and `Circle` extends `Shape`, the object is not only a `Circle` part. It contains a `Shape` part too.

- Parent part: fields and methods from `Shape`, such as `name`, `color`, and `displayInfo()`.
- Child part: fields and methods from `Circle`, such as `radius` and `area()`.

So when `c.displayInfo()` runs, Java can find the method in the parent part. When `c.area()` runs, Java finds the method in the child part.

Chapter 18 Parent Constructor Before Child Constructor

```
class A {  
    public A() {  
        System.out.println("A constructor");  
    }  
}  
  
class B extends A {  
    public B() {  
        System.out.println("B constructor");  
    }  
}  
  
B obj = new B();
```

Output:

```
A constructor  
B constructor
```

Rule. When a child object is created, the parent constructor runs first. If you do not write `super()`, Java inserts it automatically.

18.1 Why Parent First?

The parent part must exist before the child can build on it. A `CSEStudent` cannot specialize student information until the basic `Student` part exists. A `PremiumRide` cannot add premium charges until the basic `Ride` distance exists.

For tracing, this means the constructor order is never optional:

1. static fields already exist before objects;
2. parent instance fields are initialized;
3. parent constructor runs;
4. child instance fields are initialized;
5. child constructor runs.

If the parent constructor changes a static variable, the child constructor sees the changed value. This is one of the traps in advanced inheritance traces.

Chapter 19 `super`

19.1 `super()`

`super()` calls a parent constructor.

```
class Student {  
    public Student(String name) { }  
}  
  
class CSEStudent extends Student {  
    public CSEStudent(String name) {  
        super(name);  
    }  
}
```

19.2 super.method()

`super.method()` calls the parent version of a method.

```
class Parent {
    public void show() {
        System.out.println("Parent");
    }
}

class Child extends Parent {
    public void show() {
        super.show();
        System.out.println("Child");
    }
}
```

19.3 super.variable

`super.variable` accesses the parent field when the child hides a field with the same name.

Trap. `super` does not mean another object. It means the parent part of the same object.

Expression	Meaning Inside A Child Method
<code>this.x</code>	use the current object's field, usually the child field if the child hides <code>x</code>
<code>super.x</code>	use the parent field inside the same current object
<code>((Parent)this).x</code>	view the same object through the parent type, so field lookup uses parent <code>x</code>

Rule. `super.x` and `((Parent)this).x` can reach the same hidden parent field, but `super` is only available inside the child class. A cast can be written outside too if the types are valid.

19.4 The Parent Part Is Not A Separate Creature

When `B` extends `A`, a `B` object contains an `A` part. But that parent part is not another object floating somewhere else. It is inside the same object.

```
class A {
    public int sum = 1;
}

class B extends A {
    public int sum = 10;

    public void show() {
        System.out.println(this.sum);
        System.out.println(super.sum);
    }
}
```

For a `B` object:

- `this.sum` means the child field, value 10;
- `super.sum` means the parent field, value 1.

Both are inside the same `B` object. That is why updating `super.sum` changes the parent part of the same object that `this` is using.

Chapter 20 Overriding And Overloading

20.1 Overriding

```
class A {  
    public void speak() {  
        System.out.println("A");  
    }  
}  
  
class B extends A {  
    public void speak() {  
        System.out.println("B");  
    }  
}
```

Rule. Overriding means the child has a method with the same signature as the parent method.

20.2 Overloading

```
public void calculateFee() { }  
public void calculateFee(int discount) { }
```

Rule. Overloading means same method name but different parameter list.

Exam warning. Do not confuse them. Overriding is about inheritance behavior. Overloading is about argument matching.

Chapter 21 toString() And Object Printing

```
class Ride {  
    public int distance;  
    public String toString() {  
        return "Distance: " + distance + " km";  
    }  
}  
  
Ride r = new Ride();  
System.out.println(r);
```

Rule. When you print an object directly, Java calls its `toString()` method.

This explains many CSE111 outputs where the driver says:

```
System.out.println(ticket1);  
System.out.println(c1);
```

Your class must return a formatted String from `toString()`.

Part VI The Dangerous Zone: Field Hiding And Method Lookup

Chapter 22 The Hydra Of Same Names

Myth hook. The Hydra has many heads. Java tracing questions grow many variables with the same name: `x`, `y`, `sum`, `temp`. Cut the wrong head, and the output changes.

This is the part where many students say, “But I know inheritance, why am I still getting tracing wrong?” The answer is field hiding. Overriding is for methods. Fields do not override. If the child declares a field with the same name as the parent, the child does not erase the parent field. The object now has two fields with the same name in two different parts.

That is why the Hydra image matters. You may see one name, `sum`, but there may be multiple heads:

- local `sum`;
- child instance `sum`;
- parent instance `sum`;
- child static `sum`;
- parent static `sum`.

The code decides which head you are touching.

One B object in memory		
B child part / outer box	<code>B.sum = 10</code>	<code>this.sum</code> and <code>b.sum</code> point here
A parent part / inner box	<code>A.sum = 1</code>	<code>super.sum</code> and <code>((A)b).sum</code> point here

Rule. The parent part is inside the child object. It is not a second object. A cast or `super` changes which field box Java looks at inside the same object.

22.1 Field Hiding

```
class A {
    public int sum = 1;
}

class B extends A {
    public int sum = 10;
}
```

A B object now contains two `sum` fields:

- parent part: `A.sum`;
- child part: `B.sum`.

```
B b = new B();
System.out.println(b.sum);           // child sum
System.out.println(((A)b).sum);     // parent sum
```

Rule. Fields are not overridden. They are hidden.

22.2 Method Overriding Versus Field Hiding

Compare these two cases:

```
class A {
    public int x = 1;
    public void show() {
        System.out.println("A");
    }
}

class B extends A {
    public int x = 2;
    public void show() {
        System.out.println("B");
    }
}
```

Now:

```
A obj = new B();
obj.show();
System.out.println(obj.x);
```

Output:

```
B
1
```

Why?

1. `obj.show()` is a method call. Methods are polymorphic. The actual object is B, so `B.show()` runs.
2. `obj.x` is a field access. Fields are not polymorphic. The reference type is A, so `A.x` is used.

This is one of the most important pages in the whole book. If a student understands this page, polymorphism error tables become much less mysterious.

Chapter 23 Static Field Hiding

```
class A {
    public static int x = -10;
}

class B extends A {
    public static int x = 0;
}
```

```
System.out.println(A.x); // -10
System.out.println(B.x); // 0
```

Trap. If both parent and child have static x, always check whether the code says `A.x`, `B.x`, `this.x`, or just `x`.

Chapter 24 this, super, And Class Names

```
class A {
    public static int x = 1;
    public int y = 2;
}

class B extends A {
    public static int x = 10;
    public int y = 20;

    public void show() {
        System.out.println(x);           // B.x
        System.out.println(this.y);      // B object's child y
        System.out.println(super.y);     // parent y inside same object
        System.out.println(A.x);         // A static x
        System.out.println(B.x);         // B static x
    }
}
```

Rule. Use a memory table with separate columns for parent fields and child fields.

Chapter 25 Local Shadowing

```
class A {
    public static int x = 5;
    public int y = 7;

    public void methodA(int y) {
        int x = 2;
        System.out.println(x + " " + y + " " + this.y + " " + A.x);
    }
}
```

Priority:

1. local variables and parameters;
2. instance fields through `this`;
3. parent fields through `super`;
4. static fields through class name.

Exam warning. If a method has `int x = 0`, then plain `x` usually means the local variable, not the static field.

Chapter 26 Increment Traps

```
int temp = 4;
int a = temp++;
int b = ++temp;
```

26.1 Step By Step

1. Start: `temp = 4`.
2. `a = temp++`: assign old value 4 to `a`, then `temp` becomes 5.
3. `b = ++temp`: first `temp` becomes 6, then assign 6 to `b`.

Final:

```
temp = 6, a = 4, b = 6
```

Trap. In tracing questions, one `++temp` instead of `temp++` can change every later line.

Part VII Polymorphism

Chapter 27 Proteus And The Many Forms

Myth hook. Proteus can appear in many forms. A child object can stand under a parent reference. But Java still remembers the real object underneath.

Polymorphism sounds like a big word, but the idea is simple: one reference type can hold many child object types. A `GameSession` reference can point to a `RankedSession` or a `ScrimSession`. A `Notice` reference can point to an `EmailNotice` or an `AlertNotice` object if those classes are in the inheritance tree.

The hard part is that Java makes two decisions at two different times.

- At compile time, Java checks the reference type.
- At runtime, Java runs overridden methods using the actual object type.

This is why a line can fail before running, even if the object would have had the method. The compiler does not look into the future. It first asks, “Does the reference type promise this method?”

27.1 Reference Type And Object Type

```
Parent p = new Child();
```

- Parent is the reference type.
- Child is the object type.

Rule. Compile time looks at the reference type. Runtime overridden method execution looks at the actual object type.

27.2 The Two Gates

Every polymorphic method call passes two gates.

Gate	Question Java Asks
Compile-time gate	Does the reference type or its parents contain this method signature?
Runtime gate	Which overridden version should run for the actual object?

If the compile-time gate says no, the program does not compile. Runtime never gets a chance.

Rule. Divine Law Of Polymorphism: methods follow the actual object at runtime, but only after the reference type passes the compile-time check. Fields follow the reference type or casting type.

27.3 Compile-Time Versus Runtime Error Table

Situation	What Java Checks	Result
<code>p.childOnly()</code> where <code>p</code> is a parent reference	Does the parent type know <code>childOnly()</code> ?	If no, compile-time error
<code>((Child)p).childOnly()</code>	Are the types related, and does <code>Child</code> know the method?	If yes, compile passes
Bad downcast at runtime	Is the actual object really a <code>Child</code> ?	If no, runtime error
Field access <code>p.x</code>	Which field belongs to the reference type?	No runtime override for fields
Overridden method <code>p.show()</code>	Which method body belongs to the actual object?	<code>Child</code> method may run

Chapter 28 Method Lookup

```
class Parent {
    public void greet() {
        System.out.println("Parent");
    }
}

class Child extends Parent {
    public void greet() {
        System.out.println("Child");
    }
}

Parent p = new Child();
p.greet();
```

28.1 When The Reference Type Does Not Have The Method

```
class Parent {
    public void greet() {
        System.out.println("Parent");
    }
}

class Child extends Parent {
    public void secret() {
        System.out.println("Child secret");
    }
}

Parent p = new Child();
p.secret(); // compile-time error
```

The actual object is a `Child`, and `Child` really has `secret()`. But the reference type is `Parent`. Java first asks whether a `Parent` reference is allowed to call `secret()`. It is not. So the line fails at compile time.

To call it, you need a valid cast:

```
((Child)p).secret();
```

Now Java checks whether the actual object really is a `Child`. Here it is, so the method runs.

28.2 Step By Step

1. Compile time: Does Parent contain greet()? Yes.
2. Runtime: The actual object is Child.
3. Does Child override greet()? Yes.
4. Run Child.greet().

Output:

```
Child
```

Chapter 29 Variable Lookup

```
class Parent {  
    public String texture = "Parent";  
}  
  
class Child extends Parent {  
    public String texture = "Child";  
}  
  
Parent p = new Child();  
System.out.println(p.texture);
```

Output:

```
Parent
```

Rule. Variables are selected by reference type, not runtime object type.

Exam warning. This is the key lesson for polymorphism tables: methods are polymorphic, fields are not.

Chapter 30 Polymorphic Arrays

```
class Notice {  
    public String title;  
    public Notice(String title) {  
        this.title = title;  
    }  
    public int priority() {  
        return 1;  
    }  
    public void show() {  
        System.out.println("Notice: " + title);  
    }  
}  
  
class EmailNotice extends Notice {  
    public int attachments;  
    public EmailNotice(String title, int attachments) {  
        super(title);  
        this.attachments = attachments;  
    }  
    public int priority() {  
        return 1 + attachments;  
    }  
    public void show() {
```

```

        System.out.println("Email: " + title);
    }
}

class AlertNotice extends Notice {
    public int level;
    public AlertNotice(String title, int level) {
        super(title);
        this.level = level;
    }
    public int priority() {
        return level * 10;
    }
    public void show() {
        System.out.println("Alert: " + title);
    }
}

Notice[] board = new Notice[3];
board[0] = new Notice("Club meeting");
board[1] = new EmailNotice("Assignment update", 2);
board[2] = new AlertNotice("Server down", 3);

for (int i = 0; i < board.length; i++) {
    board[i].show();
    System.out.println(board[i].priority());
}

```

Rule. A parent array can store child objects. When overridden methods are called, the object type decides which version runs.

Even though the array type is `Notice[]`, the second slot still holds an `EmailNotice` object and the third slot still holds an `AlertNotice` object. So `board[i].show()` and `board[i].priority()` use the overridden child versions when the actual object is a child object.

30.1 Why This Matters

Hard coding tasks often ask for a board, desk, panel, or manager class:

- `NoticeBoard` stores `Notice` references;
- `CoursePanel` stores `Task` references;
- `RiskDesk` stores `CaseFile` references.

The board calls parent methods, but overridden child methods execute.

Chapter 31 Abstract Classes And Interfaces

Some CSE111 sections include abstract classes or interfaces near the end of inheritance and polymorphism. The tracing idea stays the same: compile time checks what the reference type promises, and runtime runs the actual object's overridden method.

31.1 Abstract Class

```

abstract class Task {
    public String title;

    public Task(String title) {
        this.title = title;
    }
}

```

```
public abstract int priority();

public void show() {
    System.out.println(title + ": " + priority());
}

}

class QuizTask extends Task {
    public int questions;

    public QuizTask(String title, int questions) {
        super(title);
        this.questions = questions;
    }

    public int priority() {
        return questions * 2;
    }
}
```

Rule. An abstract class can have fields, constructors, normal methods, and abstract methods. You cannot create new `Task()`, but a `Task` reference can point to a `QuizTask` object.

31.2 Interface

```
interface Payable {
    public double pay();
}

class TutorShift implements Payable {
    public int hours;

    public TutorShift(int hours) {
        this.hours = hours;
    }

    public double pay() {
        return hours * 500;
    }
}

Payable p = new TutorShift(3);
System.out.println(p.pay());
```

Output:

```
1500.0
```

Trap. With an interface reference, compile time allows only methods promised by the interface unless you cast. The actual object still decides the overridden method body.

Part VIII Casting And Error Traps

Chapter 32 Icarus And Dangerous Casting

Myth hook. Icarus flew with borrowed wings. Casting also borrows a type. If the object cannot truly carry that type at runtime, the flight fails.

32.1 Upcasting

```
Child c = new Child();
Parent p = c;
```

Rule. Upcasting from child to parent is safe and automatic.

32.2 Downcasting

```
Parent p = new Child();
Child c = (Child) p;
```

This is safe because the actual object is a Child.

32.3 Bad Downcasting

```
Parent p = new Parent();
Child c = (Child) p; // compiles, but runtime error
```

Trap. Compile time asks whether the types are in the same inheritance branch. Runtime asks whether the actual object really is compatible with the casting type.

32.4 A Slow Casting Story

Suppose the inheritance tree is:

Message $\leftarrow$ EmailMessage $\leftarrow$ UrgentEmailMessage

Here is the tiny family used in this section:

```
class Message {
    public String label = "Message";
    public void show() {
        System.out.println("General message");
    }
}

class EmailMessage extends Message {
    public String label = "Email";
    public void send() {
        System.out.println("Sending email");
    }
}

class UrgentEmailMessage extends EmailMessage {
    public String label = "Urgent";
    public void escalate() {
```



```
        System.out.println("Escalating");
    }
}
```

Now:

```
EmailMessage e = new UrgentEmailMessage();
```

This is safe. An `UrgentEmailMessage` is a kind of `EmailMessage`. But now ask:

```
((UrgentEmailMessage)e).escalate();
```

Step by step:

1. Compile time: `EmailMessage` and `UrgentEmailMessage` are in the same branch. Good.
2. Compile time: Does `UrgentEmailMessage` have `escalate()`? Yes.
3. Runtime: Is the actual object an `UrgentEmailMessage`? Yes, because it was created with `new UrgentEmailMessage()`.
4. Run the method.

Now change only the object:

```
EmailMessage e = new EmailMessage();
((UrgentEmailMessage)e).escalate();
```

Compile time may accept the cast because the classes are related. Runtime rejects it because the actual object is not an `UrgentEmailMessage`. This is a runtime error.

Chapter 33 Method Casting Flow

For:

```
Message message = new UrgentEmailMessage();
((UrgentEmailMessage) message).escalate();
```

Use this flow:

1. Are reference type and casting type in the same branch? If no, compile-time error.
2. Does the casting type or its parents contain the method? If no, compile-time error.
3. At runtime, can the object be treated as the casting type? If no, runtime error.
4. If yes, run the method. If overridden, object type decides the final method body.

Chapter 34 Variable Casting Flow

For:

```
Message message = new UrgentEmailMessage();
System.out.println(((EmailMessage) message).label);
```

Use this flow:

1. Are reference type and casting type in the same branch? If no, compile-time error.
2. Does the casting type or its parents contain the variable? If no, compile-time error.
3. At runtime, can the object be treated as the casting type? If no, runtime error.
4. If yes, print the variable from the casting type. If not found there, search its parents.

Here the printed value is `Email`, because the casting type is `EmailMessage`. The actual object is still an `UrgentEmailMessage`, but fields are not selected by runtime type.

Rule. Casting changes the compile-time viewpoint. It does not change the actual object.

Chapter 35 Object Reference Traps

```
Object obj = new EmailMessage();  
// obj.send(); // compile-time error because Object has no send()
```

Rule. With an `Object` reference, compile time only allows methods known in `Object`, unless you cast.

```
((EmailMessage)obj).send();
```

Now Java checks whether the actual object can be treated as `EmailMessage`.

Part IX Master Tracing

Chapter 36 Static And Reference Tracing Pattern

36.1 Trap List

Static-and-reference tracing questions test:

- static variables shared across all objects;
- static arrays shared across all objects;
- constructors changing static state;
- copy constructors calling methods;
- local variables hiding static or instance variables;
- methods mutating objects received as parameters.

36.2 Worked Pattern

```
class LedgerTrace {
    public static int total = 5;
    public int score = 2;
    public int bonus;

    public LedgerTrace() {
        bonus = total - score;
        total += 3;
    }

    public LedgerTrace(LedgerTrace t) {
        score = t.bonus;
        t.update(1);
        bonus = total + score;
    }

    public void update(int x) {
        score += x + total;
        total -= 2;
        System.out.println(score + " " + bonus + " " + total);
    }

    public static void swapBonus(LedgerTrace first, LedgerTrace second) {
        int temp = first.bonus;
        first.bonus = second.score;
        second.score = temp + total;
        System.out.println(first.bonus + " " + second.score + " " + total);
    }
}
```

Tracing move. When a constructor receives another object, draw both objects. If the constructor calls a method on the old object, update the old object first.

Tracing move. Try first: before reading the solution, write the static row, draw object a, then trace the copy constructor for object b. Only reveal the output after your table has a row for every method call.

36.3 Complete Solution Of The Pattern

Driver:

```
LedgerTrace a = new LedgerTrace();
LedgerTrace b = new LedgerTrace(a);
a.update(2);
LedgerTrace.swapBonus(a, b);
```

Trace:

1. Static starts: total = 5.
2. a = new LedgerTrace(): a.score = 2. Constructor sets a.bonus = 5 - 2 = 3, then total = 8.
3. b = new LedgerTrace(a) starts. b.score first has field value 2, then constructor sets b.score = a.bonus = 3.
4. Constructor calls a.update(1). Now a.score = 2 + 1 + 8 = 11. Then total = 6. Print 11 3 6.
5. Constructor continues: b.bonus = total + b.score = 6 + 3 = 9.
6. a.update(2): a.score = 11 + 2 + 6 = 19. Then total = 4. Print 19 3 4.
7. swapBonus(a,b): local temp = a.bonus = 3.
8. a.bonus = b.score = 3.
9. b.score = temp + total = 3 + 4 = 7.
10. Print 3 7 4.

Output:

```
11 3 6
19 3 4
3 7 4
```

The hidden danger was line 4. Creating b changed a and printed output.

Chapter 37 Basic Inheritance Tracing Pattern

37.1 Trap List

Basic inheritance tracing questions test:

- child object has parent fields;
- child method can call inherited parent method;
- plain field names may refer to inherited fields;
- static fields in parent can be changed by child code;
- super accesses parent part.

37.2 Small Pattern

```
class A {
    public int temp = 4;
    public int sum = 1;
    public int y = 2;

    public void methodA(int m, int n) {
        int x = 0;
        y = y + m + (temp++);
    }
}
```

```

        x = x + 2 + n;
        sum = sum + x + y;
        System.out.println(x + " " + y + " " + sum);
    }
}

class B extends A {
    public int x = 6;

    public void methodB(int m, int n) {
        int y = 0;
        y = ++y + this.y;
        x = this.y + 2 + temp;
        methodA(x, y);
        sum = x + y + super.sum;
        System.out.println(x + " " + y + " " + sum);
    }
}

```

Trap. Inside B, fields `temp`, `sum`, and `y` may be inherited from A. Do not assume every field is declared inside B.

Tracing move. Try first: trace `a1.methodA(1,1)` completely before touching `b1`. The two objects do not share instance fields.

37.3 Complete Solution Of The Pattern

Driver:

```

A a1 = new A();
a1.methodA(1, 1);
B b1 = new B();
b1.methodB(1, 2);

```

For `a1.methodA(1,1)`:

1. `a1.temp = 4`, `a1.sum = 1`, `a1.y = 2`.
2. Local `x = 0`.
3. `y = y + m + (temp++)` means `a1.y = 2 + 1 + 4 = 7`; then `a1.temp = 5`.
4. Local `x = 0 + 2 + 1 = 3`.
5. `sum = 1 + 3 + 7 = 11`.
6. Print 3 7 11.

For `b1.methodB(1,2)`:

1. `b1` has inherited `temp=4`, `sum=1`, `y=2`, and child field `x=6`.
2. Local `y = 0`.
3. `y = ++y + this.y`: local `y` becomes `1 + 2 = 3`.
4. Child field `x = this.y + 2 + temp = 2 + 2 + 4 = 8`.
5. `methodA(x,y)` calls inherited `A.methodA(8,3)`.
6. Inside `methodA`: inherited `y = 2 + 8 + 4 = 14`; inherited `temp = 5`; local `x = 0 + 2 + 3 = 5`; inherited `sum = 1 + 5 + 14 = 20`. Print 5 14 20.
7. Back in `methodB`: `sum = child x + local y + super.sum = 8 + 3 + 20 = 31`.

8. Print 8 3 31.

Output:

```
3 7 11
5 14 20
8 3 31
```

Chapter 38 Override, Constructor, And Hidden Field Tracing Pattern

38.1 Trap List

Advanced inheritance tracing questions test:

- parent constructor before child constructor;
- child fields hiding parent fields;
- static fields hidden in child class;
- overridden method called through `this`;
- `super.method()` forcing parent method;
- constructor calling a method before child construction is fully finished.

38.2 The Most Dangerous Pattern

```
class A {
    public A(int x) {
        this.methodA(3, 5);
    }

    public void methodA(int m, int n) {
        System.out.println("A");
    }
}

class B extends A {
    public int sum = -6;

    public B() {
        super(5);
        sum = 0;
    }

    public void methodA(int m, int n) {
        System.out.println("B " + sum);
    }
}
```

```
B b = new B();
```

Tracing move. Try first: predict the output before reading the explanation. The key question is whether `sum = -6` has already run when `B.methodA()` prints.

38.2.1 What Happens

1. Java starts creating a B object.
2. Parent constructor `A(int x)` runs first.

3. `A(int x)` calls `this.methodA(3,5)`.
4. Since the actual object is `B`, the overridden `B.methodA()` runs.
5. But `B`'s field initializer and constructor body have not run yet. The child field `sum` still has its default value `0`, not `-6`.

Exam warning. This is one of the hardest CSE111 traps. Overridden methods can run from a parent constructor.

Moment	What Has Happened	sum
Object allocated	all fields have default values	0
Parent constructor runs	<code>A(int x)</code> calls <code>this.methodA(3,5)</code>	0
Override dispatch	actual object is <code>B</code> , so <code>B.methodA()</code> runs early	0
Child initializer	after parent constructor returns, <code>sum = -6</code> runs	-6
Child constructor body	final line <code>sum = 0</code> runs	0

Output:

B 0

After that print, Java returns from the parent constructor, applies the child field initializer `sum = -6`, and then runs the child constructor body `sum = 0`. If another line printed `b.sum` after construction, it would print `0`. During the parent constructor call, it printed `0` for a different reason: the child initializer had not run yet.

Chapter 39 The Universal Tracing Checklist

Use this checklist for every tracing question:

1. Write all static variables before the driver starts.
2. For each `new`, create a new object table.
3. Run parent constructor before child constructor.
4. Run field initializers before constructor body.
5. If a constructor calls a method, trace the method immediately.
6. For every method call, create a local frame for parameters and local variables.
7. If a variable name repeats, identify whether it is local, instance, static, parent, or child.
8. For plain method calls, check overriding.
9. For `super.method()`, run parent version.
10. For fields, remember that fields are selected by reference/casting type, not runtime object.
11. Print output immediately when a print statement executes.
12. Never reset static variables unless code explicitly changes them.

Part X Practice System

Chapter 40 Easy Coding Drills

40.1 Drill 1: Static Counter

Design a `TempleVisitor` class.

```
public class Tester {  
    public static void main(String[] args) {  
        System.out.println(TempleVisitor.count);  
        TempleVisitor v1 = new TempleVisitor("Nikos");  
        TempleVisitor v2 = new TempleVisitor("Daphne");  
        v1.show();  
        v2.show();  
        System.out.println(TempleVisitor.count);  
    }  
}
```

Tracing move. Try first: write only the fields and constructor before looking at the solution. The first printed line proves that count starts at 0 before any visitor exists.

Expected output:

```
0  
Visitor ID: TV-1  
Name: Nikos  
Visitor ID: TV-2  
Name: Daphne  
2
```

40.2 Solution Plan

- count must be static.
- Each object needs its own id and name.
- Constructor increments count, then builds ID.

One possible solution:

```
class TempleVisitor {  
    public static int count = 0;  
    private String id;  
    private String name;  
  
    public TempleVisitor(String name) {  
        count++;  
        id = "TV-" + count;  
        this.name = name;  
    }  
  
    public void show() {  
        System.out.println("Visitor ID: " + id);  
        System.out.println("Name: " + name);  
    }  
}
```


Chapter 41 Medium Inheritance Drills

41.1 Drill 2: Parent Fee, Child Fee

```
class Quest {
    private String title;
    public int days;
    public Quest(String title, int days) {
        this.title = title;
        this.days = days;
    }
    public String getTitle() {
        return title;
    }
    public double reward() {
        return days * 100;
    }
}
```

Design `HeroQuest` extends `Quest`. It adds `dangerLevel`. Its reward is `super.reward() + dangerLevel * 50`.

Tracing move. Try first: write the child constructor and the overridden `reward()` method before reading the solution. Do not access private `title` directly.

41.2 What Students Must Notice

- `title` is private, so use `getTitle()`.
- `reward()` should be overridden.
- The child constructor must call `super(title, days)`.

One possible solution:

```
class HeroQuest extends Quest {
    private int dangerLevel;

    public HeroQuest(String title, int days, int dangerLevel) {
        super(title, days);
        this.dangerLevel = dangerLevel;
    }

    public double reward() {
        return super.reward() + dangerLevel * 50;
    }

    public void show() {
        System.out.println(getTitle() + ": " + reward());
    }
}
```

Chapter 42 Hard Polymorphism Drills

42.1 Drill 3: Olympus Council

```
class Petition {
    private String citizen;
    public int urgency;
    public Petition(String citizen, int urgency) {
```

```

    this.citizen = citizen;
    this.urgency = urgency;
}
public String getCitizen() {
    return citizen;
}
public int priority() {
    return urgency;
}
public void show() {
    System.out.println(citizen + ": " + priority());
}
}

```

Design:

- WarPetition extends Petition, adds threats, priority is `super.priority() + threats * 10`;
- HarvestPetition extends Petition, adds villages, priority is `super.priority() + villages * 5`;
- CouncilBoard, stores up to 5 Petition references and prints all petitions using polymorphism.

Tracing move. Try first: decide the array type in CouncilBoard before reading the solution. If the board must store different child petition objects, the array should use the parent type.

Rule. The board should store parent references but child objects. The call to `priority()` must be polymorphic.

One possible solution:

```

class WarPetition extends Petition {
    private int threats;

    public WarPetition(String citizen, int urgency, int threats) {
        super(citizen, urgency);
        this.threats = threats;
    }

    public int priority() {
        return super.priority() + threats * 10;
    }
}

class HarvestPetition extends Petition {
    private int villages;

    public HarvestPetition(String citizen, int urgency, int villages) {
        super(citizen, urgency);
        this.villages = villages;
    }

    public int priority() {
        return super.priority() + villages * 5;
    }
}

class CouncilBoard {
    private Petition[] petitions = new Petition[5];
    private int count = 0;

    public void addPetition(Petition p) {
        if (count < petitions.length) {
            petitions[count] = p;
        }
    }
}

```

```

        count++;
    }
}

public void showAll() {
    for (int i = 0; i < count; i++) {
        petitions[i].show();
    }
}
}
}

```

Chapter 43 Tracing Drill: Static And Copy Constructor

```

class Trial {
    public static int seed = 4;
    public int score = 1;
    public int sum = 0;

    public Trial() {
        score += seed;
        seed += 2;
    }

    public Trial(Trial t) {
        sum = t.score;
        t.test(2);
        score = seed + sum;
    }

    public void test(int x) {
        int score = x + seed;
        this.sum += score + this.score;
        seed--;
        System.out.println(score + " " + this.sum + " " + seed);
    }
}

```

Driver:

```

Trial t1 = new Trial();
Trial t2 = new Trial(t1);
t2.test(1);

```

Tracing move. Try first: solve the two printed lines before checking the walkthrough. The copy constructor prints once because it calls `t.test(2)`.

43.1 How To Attack

1. Static: `seed = 4`.
2. Create `t1`. Update `score` and `seed`.
3. Create `t2`. Copy constructor touches `t1`.
4. Inside `test`, local `score` hides instance `score`.
5. Print each line as soon as it happens.

Solution:

1. `t1 = new Trial(): t1.score = 1 + 4 = 5`, then `seed = 6`.

2. `t2 = new Trial(t1)` starts. `t2.sum = t1.score = 5`.
3. The copy constructor calls `t1.test(2)`. Local score = $2 + 6 = 8$.
4. `t1.sum = 0 + 8 + 5 = 13`, then `seed = 5`. Print 8 13 5.
5. Constructor continues: `t2.score = seed + t2.sum = 5 + 5 = 10`.
6. `t2.test(1)`: local score = $1 + 5 = 6$.
7. `t2.sum = 5 + 6 + 10 = 21`, then `seed = 4`. Print 6 21 4.

Output:

```
8 13 5
6 21 4
```

Chapter 44 Inheritance And Hidden Fields

```
class Parent {
    public static int x = 2;
    public int y = 3;
    public int sum = 1;

    public Parent() {
        y += x;
        x++;
    }

    public void act(int n) {
        int x = n;
        sum += x + y + Parent.x;
        System.out.println(x + " " + y + " " + sum);
    }
}

class Child extends Parent {
    public static int x = 5;
    public int y = 7;

    public Child() {
        super();
        this.y += x;
        sum += this.y;
    }

    public void act(int n) {
        super.act(n + 1);
        x += n + this.y;
        sum += x + super.y;
        System.out.println(x + " " + super.y + " " + sum);
    }
}
```

Driver:

```
Parent p = new Child();
p.act(2);
System.out.println(p.y);
```

Tracing move. Try first: draw one Child object with a parent part and a child part. Put `Parent.x` and `Child.x` in separate static boxes.

44.1 What This Tests

- parent constructor before child constructor;
- parent `x` and child `x` are different static fields;
- parent `y` and child `y` are different instance fields;
- `p.act(2)` runs child method because methods are polymorphic;
- `p.y` prints parent field because fields are not polymorphic.

Solution:

1. Before objects: `Parent.x = 2`, `Child.x = 5`.
2. `new Child()` runs the parent constructor first. Parent `y` becomes $3 + 2 = 5$, then `Parent.x = 3`.
3. Child field `y` starts at 7. Child constructor line `this.y += x` uses `Child.x`, so child `y = 7 + 5 = 12`.
4. Inherited `sum` becomes $1 + 12 = 13$.
5. `p.act(2)` runs `Child.act(2)` because methods are polymorphic.
6. `super.act(3)` prints using the parent part: local `x = 3`, parent `y = 5`, `sum = 13 + 3 + 5 + 3 = 24`.
Print 3 5 24.
7. Back in child method, `Child.x = 5 + 2 + 12 = 19`.
8. `sum = 24 + 19 + 5 = 48`. Print 19 5 48.
9. `p.y` uses the parent reference type, so it prints parent `y`, which is 5.

Output:

```
3 5 24
19 5 48
5
```

Chapter 45 Final Exam Strategy

45.1 For Coding Questions

1. Circle every constructor call in the driver.
2. Underline every static access through class name.
3. Mark every expected output line that comes from constructor, method, or `toString()`.
4. Decide fields: private, public, protected, static, instance.
5. Implement the smallest class that matches the driver exactly.
6. Check output formatting line by line.

45.2 For Tracing Questions

1. Draw static memory first.
2. Draw each object separately.
3. Split parent and child fields when names repeat.
4. Create a local frame for each method call.
5. Track `temp++` and `++temp` carefully.
6. Write output immediately.

45.3 For Polymorphism Questions

1. Identify reference type.
2. Identify object type.
3. If casting exists, identify casting type.
4. For methods, check compile time first, then runtime overriding.
5. For variables, use reference or casting type.
6. Separate compile-time error from runtime error.

Closing Oath Of Athena

I will not guess outputs.

I will track memory.

I will respect static state.

I will separate local variables from fields.

I will remember that methods are polymorphic but fields are not.

I will trace every constructor before trusting any object.

I will let the driver code speak, and I will build exactly what it asks.